on *Borrower-schema*. In the other relations involving *loan-number* (those on schemas *Loan-schema* and *Borrower-schema*), only one tuple per loan needs to appear.

Clearly, the lack of redundancy in our decomposition is desirable. The degree to which we can achieve this lack of redundancy is represented by several *normal forms*, which we shall discuss in the remainder of this chapter.

7.6 Boyce—Codd Normal Form

Using functional dependencies, we can define several *normal forms* that represent "good" database designs. In this section we cover BCNF (defined below), and later, in Section 7.7, we cover 3NF.

7.6.1 Definition

One of the more desirable normal forms that we can obtain is **Boyce–Codd normal form** (**BCNF**). A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency (that is, $\beta \subseteq \alpha$).
- α is a superkey for schema R.

A database design is in BCNF if each member of the set of relation schemas that constitutes the design is in BCNF.

As an illustration, consider the following relation schemas and their respective functional dependencies:

- *Customer-schema* = (*customer-name*, *customer-street*, *customer-city*)
  customer-name → *customer-street customer-city*
- *Branch-schema* = (*branch-name*, *assets*, *branch-city*)
  branch-name → *assets branch-city*
- *Loan-info-schema* = (*branch-name*, *customer-name*, *loan-number*, *amount*)
  loan-number → *amount branch-name*

We claim that *Customer-schema* is in BCNF. We note that a candidate key for the schema is *customer-name*. The only nontrivial functional dependencies that hold on *Customer-schema* have *customer-name* on the left side of the arrow. Since *customer-name* is a candidate key, functional dependencies with *customer-name* on the left side do not violate the definition of BCNF. Similarly, it can be shown easily that the relation schema *Branch-schema* is in BCNF.

The schema *Loan-info-schema*, however, is *not* in BCNF. First, note that *loan-number* is not a superkey for *Loan-info-schema*, since we *could* have a pair of tuples representing a single loan made to two people—for example,

(Downtown, John Bell, L-44, 1000)
(Downtown, Jane Bell, L-44, 1000)

Because we did not list functional dependencies that rule out the preceding case, *loan-number* is not a candidate key. However, the functional dependency *loan-number* $\rightarrow$ *amount* is nontrivial. Therefore, *Loan-info-schema* does not satisfy the definition of BCNF.

We claim that *Loan-info-schema* is not in a desirable form, since it suffers from the problem of *repetition of information* that we described in Section 7.2. We observe that, if there are several customer names associated with a loan, in a relation on *Loan-info-schema*, then we are forced to repeat the branch name and the amount once for each customer. We can eliminate this redundancy by redesigning our database such that all schemas are in BCNF. One approach to this problem is to take the existing non-BCNF design as a starting point, and to decompose those schemas that are not in BCNF. Consider the decomposition of *Loan-info-schema* into two schemas:

Loan-schema = (*loan-number, branch-name, amount*)
Borrower-schema = (*customer-name, loan-number*)

This decomposition is a lossless-join decomposition.

To determine whether these schemas are in BCNF, we need to determine what functional dependencies apply to them. In this example, it is easy to see that

loan-number $\rightarrow$ *amount branch-name*

applies to the *Loan-schema*, and that only trivial functional dependencies apply to *Borrower-schema*. Although *loan-number* is not a superkey for *Loan-info-schema*, it is a candidate key for *Loan-schema*. Thus, both schemas of our decomposition are in BCNF.

It is now possible to avoid redundancy in the case where there are several customers associated with a loan. There is exactly one tuple for each loan in the relation on *Loan-schema*, and one tuple for each customer of each loan in the relation on *Borrower-schema*. Thus, we do not have to repeat the branch name and the amount once for each customer associated with a loan.

Often testing of a relation to see if it satisfies BCNF can be simplified:

- To check if a nontrivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, compute α^+ (the attribute closure of α), and verify that it includes all attributes of R; that is, it is a superkey of R.
- To check if a relation schema R is in BCNF, it suffices to check only the dependencies in the given set F for violation of BCNF, rather than check all dependencies in F^+.

  We can show that if none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.

Unfortunately, the latter procedure does not work when a relation is decomposed. That is, it *does not* suffice to use F when we test a relation R_i, in a decomposition of R, for violation of BCNF. For example, consider relation schema $R\ (A, B, C, D, E)$, with functional dependencies F containing $A \rightarrow B$ and $BC \rightarrow D$. Suppose this were

decomposed into $R1(A, B)$ and $R2(A, C, D, E)$. Now, neither of the dependencies in F contains only attributes from (A, C, D, E) so we might be misled into thinking $R2$ satisfies BCNF. In fact, there is a dependency $AC \rightarrow D$ in F^+ (which can be inferred using the pseudotransitivity rule from the two dependencies in F), which shows that $R2$ is not in BCNF. Thus, we may need a dependency that is in F^+, but is not in F, to show that a decomposed relation is not in BCNF.

An alternative BCNF test is sometimes easier than computing every dependency in F^+. To check if a relation R_i in a decomposition of R is in BCNF, we apply this test:

- For every subset α of attributes in R_i, check that α^+ (the attribute closure of α under F) either includes no attribute of $R_i - \alpha$, or includes all attributes of R_i.

If the condition is violated by some set of attributes α in R_i, consider the following functional dependency, which can be shown to be present in F^+:

$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i.$$

The above dependency shows that R_i violates BCNF, and is a "witness" for the violation. The BCNF decomposition algorithm, which we shall see in Section 7.6.2, makes use of the witness.

7.6.2 Decomposition Algorithm

We are now able to state a general method to decompose a relation schema so as to satisfy BCNF. Figure 7.13 shows an algorithm for this task. If R is not in BCNF, we can decompose R into a collection of BCNF schemas $R_1, R_2, \ldots, R_n$ by the algorithm. The algorithm uses dependencies ("witnesses") that demonstrate violation of BCNF to perform the decomposition.

The decomposition that the algorithm generates is not only in BCNF, but is also a lossless-join decomposition. To see why our algorithm generates only lossless-join decompositions, we note that, when we replace a schema R_i with $(R_i - \beta)$ and (α, β), the dependency $\alpha \rightarrow \beta$ holds, and $(R_i - \beta) \cap (\alpha, \beta) = \alpha$.

```
result := {R};
done := false;
compute F+;
while (not done) do
    if (there is a schema Ri in result that is not in BCNF)
      then begin
              let α → β be a nontrivial functional dependency that holds
              on Ri such that α → Ri is not in F+, and α ∩ β = ∅;
              result := (result − Ri) ∪ (Ri − β) ∪ ( α, β);
           end
      else done := true;
```

Figure 7.13 BCNF decomposition algorithm.

We apply the BCNF decomposition algorithm to the *Lending-schema* schema that we used in Section 7.2 as an example of a poor database design:

$$\textit{Lending-schema} = (\textit{branch-name}, \textit{branch-city}, \textit{assets}, \textit{customer-name}, \textit{loan-number}, \textit{amount})$$

The set of functional dependencies that we require to hold on *Lending-schema* are

$$\textit{branch-name} \rightarrow \textit{assets branch-city}$$
$$\textit{loan-number} \rightarrow \textit{amount branch-name}$$

A candidate key for this schema is $\{\textit{loan-number}, \textit{customer-name}\}$.

We can apply the algorithm of Figure 7.13 to the *Lending-schema* example as follows:

- The functional dependency

  $$\textit{branch-name} \rightarrow \textit{assets branch-city}$$

  holds on *Lending-schema*, but *branch-name* is not a superkey. Thus, *Lending-schema* is not in BCNF. We replace *Lending-schema* by

  $$\textit{Branch-schema} = (\textit{branch-name}, \textit{branch-city}, \textit{assets})$$
  $$\textit{Loan-info-schema} = (\textit{branch-name}, \textit{customer-name}, \textit{loan-number}, \textit{amount})$$

- The only nontrivial functional dependencies that hold on *Branch-schema* include *branch-name* on the left side of the arrow. Since *branch-name* is a key for *Branch-schema*, the relation *Branch-schema* is in BCNF.

- The functional dependency

  $$\textit{loan-number} \rightarrow \textit{amount branch-name}$$

  holds on *Loan-info-schema*, but *loan-number* is not a key for *Loan-info-schema*. We replace *Loan-info-schema* by

  $$\textit{Loan-schema} = (\textit{loan-number}, \textit{branch-name}, \textit{amount})$$
  $$\textit{Borrower-schema} = (\textit{customer-name}, \textit{loan-number})$$

- *Loan-schema* and *Borrower-schema* are in BCNF.

Thus, the decomposition of *Lending-schema* results in the three relation schemas *Branch-schema*, *Loan-schema*, and *Borrower-schema*, each of which is in BCNF. These relation schemas are the same as those in Section 7.5, where we demonstrated that the resulting decomposition is both a lossless-join decomposition and a dependency-preserving decomposition.

The BCNF decomposition algorithm takes time exponential in the size of the initial schema, since the algorithm for checking if a relation in the decomposition satisfies BCNF can take exponential time. The bibliographical notes provide references to an

algorithm that can compute a BCNF decomposition in polynomial time. However, the algorithm may "overnormalize," that is, decompose a relation unnecessarily.

7.6.3 Dependency Preservation

Not every BCNF decomposition is dependency preserving. As an illustration, consider the relation schema

$$\textit{Banker-schema} = (\textit{branch-name, customer-name, banker-name})$$

which indicates that a customer has a "personal banker" in a particular branch. The set F of functional dependencies that we require to hold on the *Banker-schema* is

$$\textit{banker-name} \rightarrow \textit{branch-name}$$
$$\textit{branch-name customer-name} \rightarrow \textit{banker-name}$$

Clearly, *Banker-schema* is not in BCNF since *banker-name* is not a superkey.

If we apply the algorithm of Figure 7.13, we obtain the following BCNF decomposition:

$$\textit{Banker-branch-schema} = (\textit{banker-name}, \textit{branch-name})$$
$$\textit{Customer-banker-schema} = (\textit{customer-name}, \textit{banker-name})$$

The decomposed schemas preserve only $\textit{banker-name} \rightarrow \textit{branch-name}$ (and trivial dependencies), but the closure of $\{\textit{banker-name} \rightarrow \textit{branch-name}\}$ does not include $\textit{customer-name branch-name} \rightarrow \textit{banker-name}$. The violation of this dependency cannot be detected unless a join is computed.

To see why the decomposition of *Banker-schema* into the schemas *Banker-branch-schema* and *Customer-banker-schema* is not dependency preserving, we apply the algorithm of Figure 7.12. We find that the restrictions F_1 and F_2 of F to each schema are:

$F_1 = \{\textit{banker-name} \rightarrow \textit{branch-name}\}$
$F_2 = \emptyset$ (only trivial dependencies hold on *Customer-banker-schema*)

(For brevity, we do not show trivial functional dependencies.) It is easy to see that the dependency *customer-name branch-name* $\rightarrow$ *banker-name* is not in $(F_1 \cup F_2)^+$ even though it *is* in F^+. Therefore, $(F_1 \cup F_2)^+ \neq F^+$, and the decomposition is not dependency preserving.

This example demonstrates that not every BCNF decomposition is dependency preserving. Moreover, it is easy to see that *any* BCNF decomposition of *Banker-schema* must fail to preserve *customer-name branch-name* $\rightarrow$ *banker-name*. Thus, the example shows that we cannot always satisfy all three design goals:

1. Lossless join
2. BCNF
3. Dependency preservation